

Movie Recommendation System

Updated Project Architecture, File Structure and Implementation Plan

Scope: Hybrid ML + TMDB Movie Recommendation and Discovery System without Authentication

1. Purpose of This Document

This document summarizes the required updates to the existing Movie Recommendation System. It explains the recommended file structure, changes to existing files, new files to add, files/features that are not needed, and the overall system architecture.

2. Main Updates Required

- Remove authentication, login, signup, Google OAuth, user profiles and user database requirements.
- Remove hardcoded Top 10 recommendation limits and support all available similarity results using pagination or Load More.
- Support local dataset movies and movies outside the local dataset.
- Use TMDB for external movie search, details, credits, trailers and discovery.
- Add complete movie details pages and trailer viewing.
- Add genre, language, Indian and international movie discovery.
- Improve caching, retries and API/network error handling.
- Make the Backend URL setting optional, advanced and removable.
- Redesign the frontend for better responsiveness and efficient space usage.

3. Final Recommended Project Structure

```
MOVIE RECOMMENDATION SYSTEM/
■
■ ■ ■ ■ .env
■ ■ ■ ■ .gitignore
■ ■ ■ ■ requirements.txt
■ ■ ■ ■ README.md
■ ■ ■ ■ main.py
■
■ ■ ■ ■ .vscode/
■ ■ ■ ■ settings.json
■
■ ■ ■ ■ backend/
■ ■ ■ ■ __init__.py
■ ■ ■ ■ config.py
■ ■ ■ ■ recommender.py
■ ■ ■ ■ feature_builder.py
■ ■ ■ ■ tmdb_service.py
■ ■ ■ ■ movie_service.py          # NEW
■ ■ ■ ■ discovery_service.py     # NEW
■ ■ ■ ■ cache_service.py
■
■ ■ ■ ■ data/
■ ■ ■ ■ Movies Metadata.csv
■
■ ■ ■ ■ models/
■ ■ ■ ■ Df.pkl
■ ■ ■ ■ Indices.pkl
■ ■ ■ ■ TFIDF.pkl
■ ■ ■ ■ TFIDF_Matrix.pkl
■
■ ■ ■ ■ frontend/
■ ■ ■ ■ index.html
```

```

■   ■■■ pages/
■   ■   ■■■ movie-details.html    # NEW
■   ■■■ js/
■   ■   ■■■ api.js
■   ■   ■■■ app.js
■   ■   ■■■ recommendations.js
■   ■   ■■■ movie-details.js
■   ■   ■■■ discovery.js
■   ■   ■■■ settings.js
■   ■■■ css/
■   ■   ■■■ main.css
■   ■   ■■■ components.css
■   ■   ■■■ movie-card.css
■   ■   ■■■ responsive.css
■   ■■■ assets/
■   ■■■ images/
■   ■■■ icons/
■
■■■ notebook/
■   ■■■ Movie_Recsys_ML_NLP.ipynb
■
■■■ .venv/

```

4. Existing Files That Need Changes

File	Changes Required
main.py	Major update. Add routes for recommendations, search, movie details, trailers and discovery.
backend/recommender.py	Major update. Remove Top 10 restriction, calculate all similarity scores, add pagination and multiple similarity dir
backend/feature_builder.py	Improve features such as genre, overview, keywords, cast, director and language where data is available.
backend/tmdb_service.py	Major update for search, details, credits, trailers and robust TMDB communication.
backend/cache_service.py	Improve caching for search, details, discovery and trailers. Add retry/fallback behavior.
backend/config.py	Centralize environment variables, API URL, timeout and cache settings.
frontend/index.html	Major UI redesign with search, discovery, filters and efficient layout.
frontend/script.js	Refactor existing logic into modular JavaScript files.
frontend/style.css	Refactor styles into main, components, movie-card and responsive CSS files.
.env	Keep TMDB configuration and other environment variables. Never commit secrets.
requirements.txt	Keep only packages required by FastAPI, ML, HTTP requests and configuration.

5. New Backend Files and Purpose

backend/movie_service.py

Acts as the hybrid bridge between the local ML dataset and TMDB. It decides whether a movie exists locally and coordinates local or external recommendation logic.

backend/discovery_service.py

Handles trending, popular, top-rated, genre-based, language-based, Indian and international movie discovery.

6. New Frontend Files and Purpose

File	Purpose
pages/movie-details.html	Dedicated page for complete movie information and trailer viewing.
js/api.js	Central place for backend URL management and frontend API requests.
js/app.js	Application initialization and shared UI behavior.
js/recommendations.js	Recommendation rendering, similarity results and Load More behavior.

File	Purpose
js/movie-details.js	Fetch and render complete movie details, cast and trailers.
js/discovery.js	Trending, popular, genre, language and regional discovery sections.
js/settings.js	Advanced backend URL settings with Save, Test and Remove.
css/main.css	Global layout, typography and navigation.
css/components.css	Reusable buttons, chips, modals and loaders.
css/movie-card.css	Movie card styling and interactions.
css/responsive.css	Responsive layouts for desktop, tablet and mobile.

7. Files That Should Remain Mostly Untouched

- models/Df.pkl: Existing processed movie dataframe artifact.
- models/Indices.pkl: Movie title/index mapping.
- models/TFIDF.pkl: Existing TF-IDF vectorizer/model artifact.
- models/TFIDF_Matrix.pkl: Existing local similarity matrix.
- data/Movies Metadata.csv: Original dataset source.
- notebook/Movie_Recsys_ML_NLP.ipynb: Training and preprocessing notebook.

These should only be changed when intentionally retraining or improving the ML model.

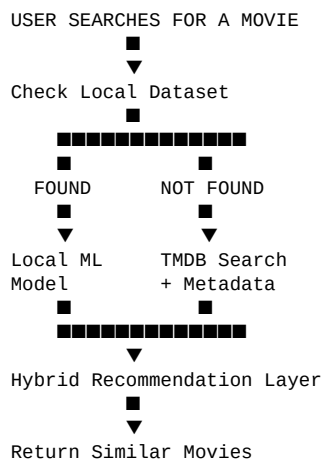
8. Files and Features Not Needed

- Authentication system
- Login and signup pages
- Google OAuth
- JWT tokens
- Password hashing and authentication security modules
- User profile pages
- User database used only for authentication

NOT REQUIRED:
backend/auth.py
backend/database.py
backend/models.py
backend/security.py

frontend/pages/login.html
frontend/pages/signup.html
frontend/pages/profile.html
frontend/js/auth.js
frontend/css/auth.css

9. Hybrid Recommendation Architecture



The system should calculate all available similarity scores but return results in pages or batches. For example, show 20 results initially and allow Load More.

10. Similarity Improvements

Overall Similarity
■■■ Genre Similarity
■■■ Overview / Content Similarity
■■■ Keyword Similarity
■■■ Cast Similarity
■■■ Director Similarity

■■■ Language Similarity

Only expose a similarity dimension when the necessary metadata exists. Overall similarity remains the primary ranking mechanism.

11. Movie Details and Trailer Requirements

- Poster and backdrop
- Title and original title
- Tagline and full overview
- Release date and runtime
- Genres, ratings and popularity
- Languages and countries
- Production information
- Cast and crew
- Trailer retrieval through TMDB video information

12. Discovery and Segregation

- Trending movies
- Popular movies
- Top-rated movies
- Movies by genre
- Movies by language
- Indian cinema
- International cinema

Genre and language should be separate filters. Indian and international categories should be based on explicit region, country or production metadata.

13. Backend URL Settings Improvement

DEFAULT:
http://127.0.0.1:8000

SETTINGS

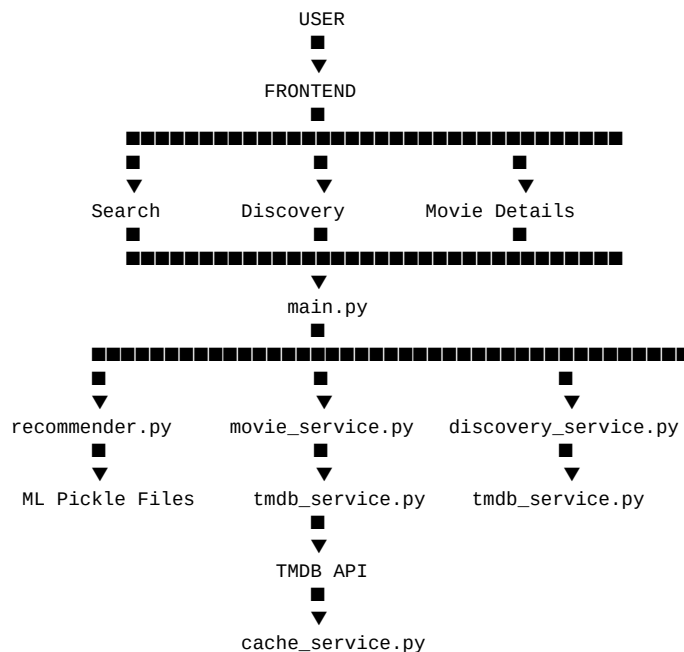
-
- Appearance
-
- Advanced Settings
 -
 - Custom Backend URL
 - Save
 - Test Connection
 - Remove Custom URL

- Do not show the backend URL box permanently.
- Show it only inside Advanced Settings.
- Save custom URLs using localStorage.
- Allow testing before use.
- Allow removal of the custom URL.
- Fall back automatically to the default backend URL.

14. API and Network Error Handling

- Use request timeouts.
- Use retry logic for temporary failures.
- Cache successful TMDb responses.
- Handle 404, 429, 5xx and network errors separately.
- Return user-friendly error messages.
- Provide frontend retry/fallback behavior.
- Do not confuse missing local dataset movies with API failures.

15. Overall Architecture



16. Recommended Implementation Order

- Phase 1: Inspect existing files and preserve currently working functionality.
- Phase 2: Improve `recommender.py` and remove Top 10 restrictions.
- Phase 3: Improve `tmdb_service.py` for search, details, credits, trailers and errors.
- Phase 4: Add `movie_service.py` for local plus external hybrid logic.
- Phase 5: Add `discovery_service.py` for discovery categories.
- Phase 6: Refactor frontend JavaScript and CSS into modules.
- Phase 7: Redesign the UI and add the movie details page.
- Phase 8: Test local movies, external movies, API failures, network failures and responsive layouts.

17. Final Summary

Keep and improve: `main.py`, `recommender.py`, `feature_builder.py`, `tmdb_service.py`, `cache_service.py`, `config.py`, `index.html`, existing frontend logic and styles through refactoring.

Add: `movie_service.py`, `discovery_service.py`, `movie-details.html` and modular JavaScript/CSS files.

Do not modify unless retraining: pickle model files, source dataset and ML notebook.

Do not add: authentication, login, signup, Google OAuth, JWT, user database or security modules.

The final project remains a focused Hybrid Movie Recommendation and Discovery System combining local ML-based similarity with TMDb-powered external discovery and detailed movie information.